# ONEAI – YOUR ULTIMATE AI CREATIVITY HUB

***A Project Report submitted in partial fulfilment of the requirements for the award of the degree of***

## Bachelor of Technology

**in**

## *Computer Science and Engineering*


| | |
|---|---|
| Akshay Agarwal | 2215000163 |
| Harendra Tyagi | 2215000718 |
| Arpan Verma | 2215000344 |

Under the Guidance of

Dr. Asheesh Tiwari

Department of Computer Engineering & Applications

**Institute of Engineering & Technology**

**GLA University**

**Mathura- 281406, INDIA**

# CONTENTS


Declaration 4

Certificate 5

Acknowledge 6

Abstract 7

List of figures 8

List of Tables 9

**CHAPTER 1 Introduction**

1.1 Motivation and Overview 10

1.2 Objectives 11

1.3 Summary of Similar Application 12

1.4 Gap Identified 13

**CHAPTER 2 Software Requirement Analysis**

2.1 Requirements Analysis 15

2.2 Feasibility Analysis 18

2.3 Modules Description 19

2.4 Use Case Scenarios 22

**CHAPTER 3 Software Design**

3.1 Sequence Diagram 25

3.2 Razorpay Working Flowchart 26

**CHAPTER 4 Implementation and User Interface**

4.1 Implementation Overview 28

4.2 Technology Stack 29

4.3 Backend Implementation 32

4.4 Frontend Implementation

**CHAPTER 5 Software Testing**

5.1 Testing Objectives 38

5.2 Testing Techniques Used 40

5.3 Testing Environment

**CHAPTER 6 Conclusion** 46

**CHAPTER 7 Summary** 48

**APPENDICES**

Sample References

# Declaration

I hereby declare that the work which is being presented in the B.Tech. Project **"ONEAI – YOUR ULTIMATE AI CREATIVITY HUB "**, in partial fulfillment of the requirements for the award of the ***Bachelor of Technology*** in Computer Science and Engineering and submitted to the Department of Computer Engineering and Applications of GLA University, Mathura, is an authentic record of my own work carried under the supervision of **Dr. Asheesh Tiwari,** Assistant Professor, Department of Computer Engineering & Applications.

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign _____

Name of Student: Akshay Agarwal

University Roll No.:2215000163

Sign _____

Name of Student: Harendra Tyagi

University Roll No.:2215000718

Sign _____

Name of Student: Arpan Verma

University Roll No.:2215000344

# CERTIFICATE

This is to certify that the above statements made by the candidate are correct to the best of my/our knowledge and belief.

_____

**Supervisor**

**Dr. Asheesh Tiwari**

Assistant Professor

Dept. of Computer Engg, & App.

_____

**Project Co-ordinator**

**(Dr. Mayank Srivastava)**

Associate Professor

Dept. of Computer Engg, & App.

_____

**Program Co-ordinator**

(**Dr. Nikhil Govil**)

Associate Professor

Dept. of Computer Engg, & App.

**Date:**

# ACKNOWLEDGEMENT

The successful completion of this project, "ONEAI – Your Ultimate AI Creativity Hub" would not have been possible without the support, guidance, and encouragement of several individuals.

We express our deep sense of gratitude to our project supervisor, **Dr. Asheesh Tiwari**, for his invaluable guidance, constructive feedback, and continuous motivation throughout the duration of this project. His insights and technical expertise helped us overcome many challenges and strengthened the quality of our work.

We would like to thank the **Department of Computer Engineering & Applications** for providing the necessary infrastructure, laboratory facilities, and academic environment that enabled us to conduct research and development activities smoothly.

We also acknowledge all faculty members for their continuous encouragement and support during our academic journey, which significantly contributed to our learning and growth.

We extend our heartfelt appreciation to our friends, peers, and family members whose constant encouragement and understanding gave us the confidence to complete this project successfully.

Finally, we thank everyone who directly or indirectly contributed to the success of this project.

Sign _____

Name of Candidate: Akshay Agarwal

University Roll No: 2215000163

Sign _____

Name of Candidate: Harendra Tyagi

University Roll No: 2215000718

Sign _____

Name of Candidate: Arpan Verma

University Roll No: 2215000344

# ABSTRACT

In an era where Artificial Intelligence (AI) has become a cornerstone of digital productivity, the landscape of AI-driven tools remains highly fragmented and financially prohibitive for the average user. While specialized models for generative art, natural language processing, and educational automation are abundant, they are typically locked behind siloed subscription models. This "subscription fatigue" creates a significant barrier to entry for students, independent creators, and smallscale developers who require multi-modal capabilities without the burden of multiple recurring monthly fees.

This report introduces OneAI a unified, multi-modal AI ecosystem designed to centralize diverse generative and analytical tasks into a single, accessible interface. By integrating state-of-the-art APIs—including Runware for high- fidelity image generation (Imagify), Hugging Face for neural text summarization (Summarify), and Google's Gemini for contextual content creation (Blogify) and automated study-aid generation (RecallAI)—the platform offers a comprehensive suite of tools powered by a robust backend.

Through technical analysis of API orchestration, system latency, and usercentric design, this research evaluates the effectiveness of a unified AI portal in reducing workflow friction. The findings suggest that OneAI provides a scalable and sustainable alternative to fragmented SaaS models, fostering a more inclusive digital environment where advanced AI tools are accessible based on utility rather than financial privilege.

# List of Figures


| Figure name | Page No. |
|---|---|
| 1. (**Fig 3.1**) Sequence Diagram depicting the full working of the project | 25 |
| 2. (**Fig 3.3**) Razorpay Working Flowchart depicting the full working of payment gateway | 26 |
| 3. (**Fig 3.4**) Agile Scrum Framework applied to OneAI development | 27 |
| 4. (**Fig 4.1**) Software Stack and API Versions | 30 |

# List of Tables


| Figure name | Page No. |
|---|---|
| 1. (**Fig 2.1**) Comparison of existing AI Subscription costs vs OneAI Model | 14 |
| 2. (**Fig 4.2**) Performance Benchmarking: Average generation Time per Task | 35 |
| 3. (**Fig 6.1**) Unit Testing : Backend API Route validation | 45 |

# Chapter 1
# Introduction

## 1.1 Motivation and Overview

This project introduces an AI-powered SaaS platform that unifies multiple intelligent tools—such as text-to-image generation, notes summarization, flashcard creation, and automated blog writing—into one streamlined web application. Instead of accessing scattered AI services across different platforms, users can perform all major productivity and creativity tasks from a single interface. The system is built using **React.js** for the user interface, **Node.js + Express.js** for backend operations, and **MongoDB** for secure and scalable data storage.

The platform integrates external generative AI APIs to perform computational tasks and uses a user authentication system to manage accounts, usage history, and available credits. A key component of the system is its **credit-based usage model**, which governs every AI operation and ensures that resource consumption is predictable, controlled, and aligned with the project's operational cost. Through Razorpay integration, users can securely purchase credits, enabling a seamless monetization flow suitable for real-world SaaS deployment.

The motivation behind building this platform stems from the increasing dependence on digital tools for learning, content creation, and productivity. Users today often switch between multiple AI applications—one for generating images, another for writing, another for summarizing text—which creates friction, inconsistency, and wasted time. By combining all major AI functions into one unified system, the project seeks to eliminate fragmentation and offer a smooth, efficient, and cohesive workflow that enhances user experience.

A major motivational factor is the need for **affordable and accessible AI tools**, especially for students and professionals who cannot afford high recurring subscription fees. This is where the **credit-based system becomes critically important**. Instead of

charging users monthly fees or forcing them into fixed plans, the platform allows them to pay *only for what they use*. This ensures fairness, cost-efficiency, and high user accessibility. The credit model also protects the developers from unpredictable API expenses by ensuring every AI call is accounted for and balanced against real operational costs.

In essence, the project is motivated by three core goals:

- **Unification** of essential AI tools into one platform
- **Affordability** through a flexible, user-friendly credit system
- **Scalability** through a monetization model that supports long-term growth

By addressing these needs, the platform positions itself as a practical and sustainable solution for modern learners, creators, and professionals who rely on AI to boost productivity.

## 1.2 Objectives

The primary objective of this project is to develop a **unified AI-powered SaaS platform** that brings together multiple intelligent tools—text-to-image generation, blog writing, notes summarization, and flashcard creation—within a single, cohesive environment. The system aims to remove the fragmentation caused by switching between multiple applications and instead offer users a seamless workflow for creativity, learning, and productivity.

A major objective of the project is the implementation of a **credit-based usage model**, which serves as the financial and operational backbone of the platform. Instead of locking users into fixed subscriptions, the credit system ensures **fair, transparent, and usage-driven access** to AI services. This model allows users to pay only for the operations they consume, making the platform far more affordable and scalable. It also gives developers precise control over API usage costs, ensuring that every AI call is balanced against revenue, preventing unexpected operational losses.

Additionally, the project aims to ensure:

- **Secure and reliable user authentication**, enabling personalized accounts and

activity tracking.

- **Scalable backend architecture** using Node.js and MongoDB to support real-world deployment.
- **Efficient integration of AI APIs**, enabling smooth execution of generative tasks.
- **A clean and intuitive user interface**, allowing users to interact effortlessly with advanced AI tools.
- **A sustainable business model**, driven by credit purchases through Razorpay integration.

Overall, the objective is to create a platform that is **technically robust, economically viable, and highly accessible**, delivering meaningful AI assistance to students, professionals, and creators.

## 1.3 Summary of Similar Application

Several AI-powered platforms are available today that offer individual or partially integrated productivity tools. These applications demonstrate the growing demand for AI-driven writing, summarization, and creative generation. However, most existing solutions focus on a **single core capability** or offer **limited tool combinations**, which leaves usability gaps and forces users to switch between multiple services. Below are some of the most relevant similar applications in the market.

- **Notion AI**

Notion AI enhances the Notion workspace by offering features such as text summarization, idea generation, rewriting assistance, and task automation. While it provides strong productivity tools, its AI features operate inside Notion only and lack external utilities such as text-to-image generation or automated flashcard creation. Users must still rely on additional tools for creative or visual tasks, limiting its scope as a complete AI productivity suite.

- **Canva AI / Canva Magic Studio**

Canva's AI tools allow users to generate images, edit visuals, and create quick content drafts. The platform is widely used for visual design, but its AI abilities are mostly limited to the creative domain. It lacks advanced text summarization, flashcard generation, or structured blog-writing tools. Canva focuses heavily on design, not on comprehensive productivity or educational workflows.

- **ChatPDF / Summarization Tools**

Websites like ChatPDF, Scholarcy, and other summarization apps allow users to upload documents and receive condensed summaries. These tools are useful for students and researchers but operate within a **single functionality** only. They do not generate blogs, flashcards, or images, and therefore cannot serve as a full productivity platform.

- **Flashcard Apps (Quizlet, Anki)**

Apps such as Quizlet and Anki are widely used for spaced repetition and study aids. They enable users to create and practice flashcards effectively. However, they lack automated AI-powered flashcard generation, advanced summarization features, or creative content generation. They serve a specific educational purpose but do not provide a multipurpose AI ecosystem.

## 1.4 Gap Identified

The major gap in all these similar applications is that **no single platform integrates all essential AI tools — image generation, blogging, note summarization, and flashcard creation — into one unified system**. Users are forced to depend on multiple applications to complete tasks across creativity, learning, and productivity domains.

Additionally, most platforms operate on monthly subscriptions, which are often **expensive** and **not usage-friendly**, especially for students. This creates a strong case for a **credit-based, multi-tool AI system** that is affordable and accessible.

**COMPARISON OF EXISTING AI SUBSCRIPTION COSTS VS. ONEAI MODEL**

| PLATFORM | PRIMARY AI MODALITY | ACCESS MODEL | STANDARD MONTHLY COST | ANNUAL COST (ESTIMATED) | KEY LIMITATION FOR STUDENTS |
|---|---|---|---|---|---|
| **OpenAI** (ChatGPT Plus) | General LLM / GPT-4o | Subscription | $20.00 (~₹1,670) | $240.00 | High entry barrier; no 'pay-only-for-what-you-use' option. |
| **Midjourney / Midjourney Pro** | High-Fidelity Images | Subscription | $10.00 (~₹835) | $120.00 | Locked into a single modality; requires separate discord integration. |
| **QuillBot / Grammarly** | Text Summarization | Subscription | $9.95 (~₹830) | $119.40 | Specialized only in text; no image or educational flashcard tools. |
| **Claude** (Anthropic) | Coding / Analysis | Subscription | $20.00 (~₹1,670) | $240.00 | Expensive for occasional users or researchers. |
| **OneAI (Proposed)** | **Multi-Modal (Gemini, Runware, Hugging Face)** | **Credit-Based (Pay-As-You-Go)** | **₹100 - ₹500 (User-Defined)** | **Variable (Usage-Dependent)** | **Democratized access; multiple tools under one wallet.** |

**Fig 2.1** Comparison of existing AI Subscription costs vs OneAI Model

This infographic effectively visualizes the **financial "Silo" effect** of the current AI market. It explicitly shows how a user would need to pay separate, high-cost monthly fees to access the same generative and analytical capabilities that your platform, **OneAI**, consolidates into a single, user-defined wallet.

The contrast between the rigid $20/month structure and your flexible ₹100-₹500 "Pay-As-You-Go" model is immediately clear.

# Chapter 2

# Software Requirement Analysis

This chapter presents a detailed analysis of the software requirements for the AI-powered productivity and creativity platform. It includes requirement gathering, feasibility study, module breakdown, functionalities, and use-case modeling. The goal is to ensure that the system is fully defined, feasible to implement, and aligned with user needs and project objectives.

## 2.1 Requirement Analysis

Requirement analysis involves identifying what the system must accomplish, who the users are, and how the platform should behave. Requirements are classified into **Functional Requirements** and **Non-Functional Requirements**.

---

### 2.1.1 Functional Requirements

The system must provide the following functionalities:

**1. User Management**

- User registration and secure login
- Email/password authentication
- User dashboard for profile details
- Display and tracking of credits
- Purchase credits using payment gateway

**2. AI Tools Integration**

Each AI module must interact with external APIs to perform its designated task.

- **Text-to-Image Generator**: Accept text prompts → Send to API → Display generated image → Allow download
- **Blog Generator**: Convert short prompts into long-form blog content
- **Flashcard Generator**: Extract key points and generate flashcards automatically
- **Notes Summarizer**: Summaries large notes into shorter, concise content

**3. Credit-Based Usage System**

- Deduct credits based on the selected tool and task size
- Prevent tool usage when credits are insufficient
- Display available credits in real-time
- Allow users to buy credits via Razorpay

**4. Payment Integration**

- Razorpay checkout integration
- Handling of payment success/failure
- Updating credit balance automatically

**5. Data Storage & Management**

- Store user profiles, payment records, summaries, blogs, flashcards, and generation history
- Maintain secure database access via MongoDB

### 2.1.2 Non-Functional Requirements

**1. Performance Requirements**

- API responses should be processed within 2–5 seconds for text tasks
- Image generation should complete within 10 seconds
- UI should remain responsive during heavy operations

**2. Security Requirements**

- Passwords must be hashed before storage
- API keys must be stored securely in the backend
- Razorpay payment gateway must follow secure transaction protocols

**3. Scalability Requirements**

- System should handle increasing users without performance degradation
- Database schema must support growing user data

**4. Usability Requirements**

- Clean and intuitive UI
- Simple navigation for all tools
- Clear credit usage display

**5. Reliability Requirements**

- The system should be able to recover from backend restarts
- Payment failures must not cause credit duplication or loss

## 2.2 Feasibility Analysis

Feasibility analysis evaluates whether the proposed system is technically, economically, and operationally achievable.

### 2.2.1 Technical Feasibility

- React.js provides a fast and user-friendly frontend
- Node.js allows asynchronous handling of API calls efficiently
- MongoDB ensures flexible and scalable storage
- External AI APIs (like Stability, Cohere, Vertex AI) reduce complexity by handling heavy ML tasks
  → **Conclusion:** Technically feasible due to modern, widely supported technologies.

### 2.2.2 Operational Feasibility

- Easy-to-use UI ensures users require minimal training
- Credit-based model ensures simple cost control for both users and admin
- System workflows mirror common productivity tool patterns
  → **Conclusion:** Operationally feasible for real-world deployment.

### 2.2.3 Economic Feasibility

- API usage costs are controlled through the credit system
- No need to train or host your own models, reducing infrastructure costs
- Scalable pricing packs make it affordable for students and professionals
  → **Conclusion:** Economically viable as cost grows only with user demand.

### 2.2.4 Time Feasibility

- Modular structure allows parallel development of features
- APIs reduce development complexity

  → **Conclusion:** Time-feasible within planned development cycles.

## 2.3 Modules Description

Your platform consists of **five major modules**:

---

### 1. User Module

- Registration & login
- Profile management
- Credits dashboard
- Payment history
- Authentication

---

### 2. AI Tools Module

Contains four sub-modules:

**a. Text-to-Image Generator**

- Takes user prompt
- Sends request to API
- Returns generated image
- Allows download

**b. Blog Generator**

- Takes topic prompt
- Generates structured blog content
- Allows user to copy or download

**c. Flashcard Generator**

- Accepts educational notes
- Extracts key points
- Converts into flashcards

**d. Notes Summarizer**

- Takes text input or file
- Produces short, concise summary

---

### 3. Credit Management Module

- Tracks user credits
- Deducts credits per tool usage
- Restricts usage if credits are insufficient
- Updates balance after purchases

### 4. Payment Module

- Razorpay payment checkout
- Secure transaction processing
- Auto-credit assignment on success

---

### 5. Database Module

- Stores user details
- Stores generated content history
- Stores credit logs and payment records

---

## 2.4 Use Case Scenarios

### Use Case 1: User Generates an Image

1. User logs in
2. Opens Text-to-Image tool
3. Enters prompt
4. System checks credit balance
5. Credits deducted → API call → Image generated
6. User downloads the image

---

### Use Case 2: User Summarizes Notes

1. User uploads text
2. System checks available credits
3. Summary is generated
4. Credits deducted
5. User saves summary

---

### Use Case 3: User Purchases Credits

1. User opens "Buy Credits"
2. Selects credit pack
3. Payment through Razorpay
4. On success → Credits added
5. User receives receipt

### Use Case 4: Blog Post Generation

1. User enters blog topic
2. System processes request
3. AI returns long-form blog
4. Credits deducted
5. User copies or downloads the content

---

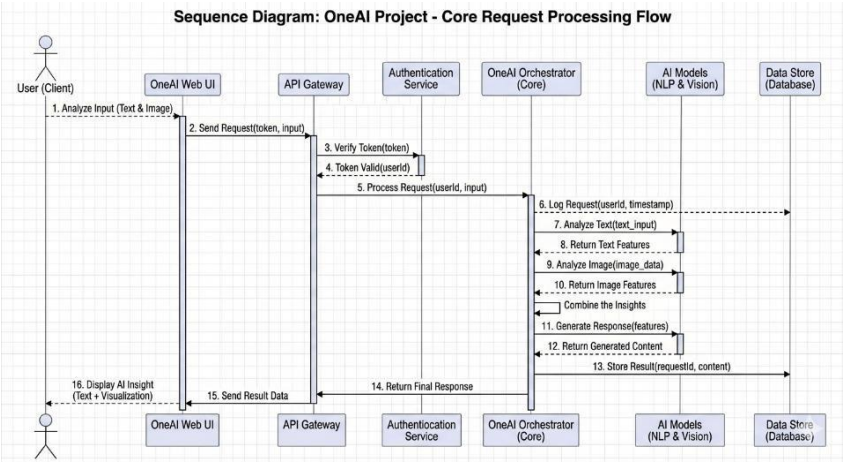
### Use Case 5: Flashcard Creation

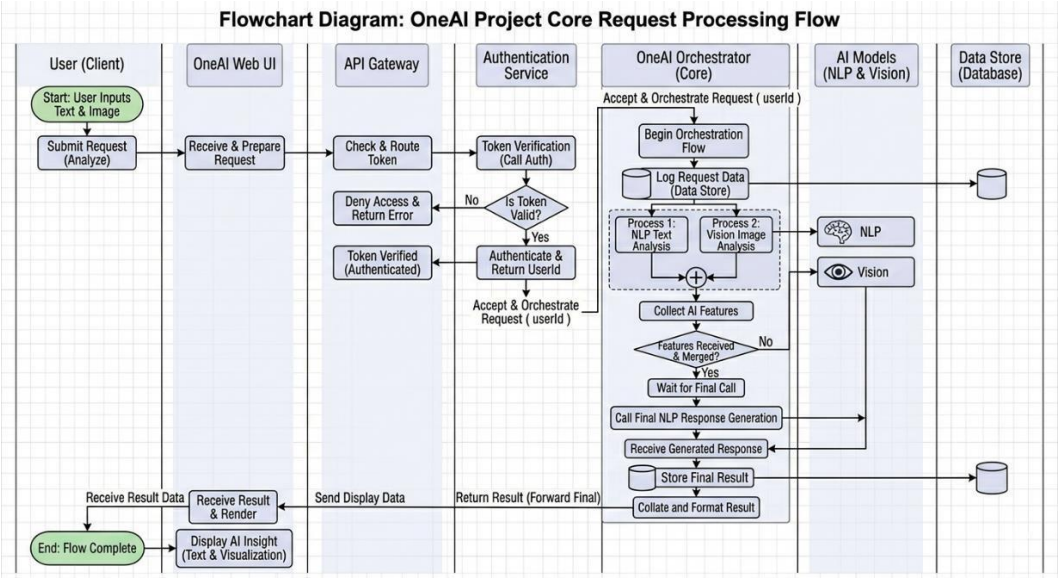
1. User uploads study notes
2. System extracts key points
3. AI generates flashcards
4. Credits deducted
5. Flashcards displayed to user

# Chapter 3

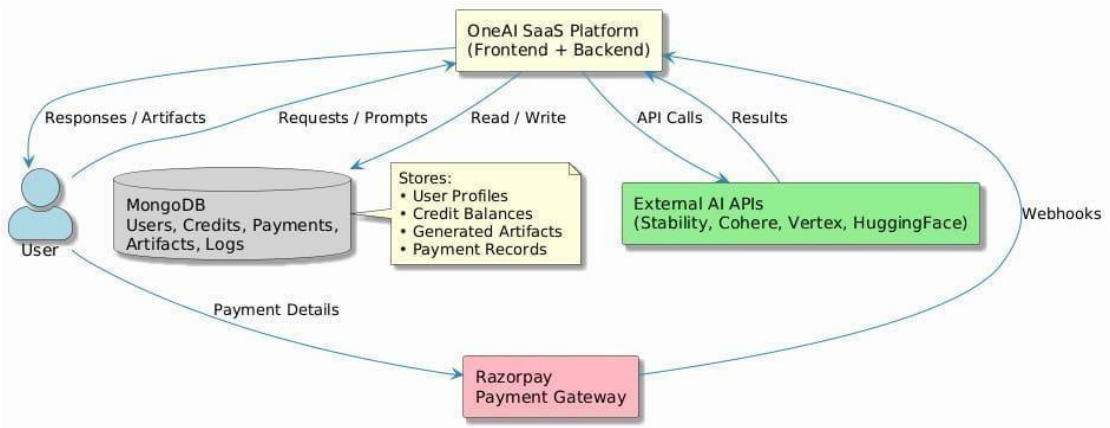
# Software Design

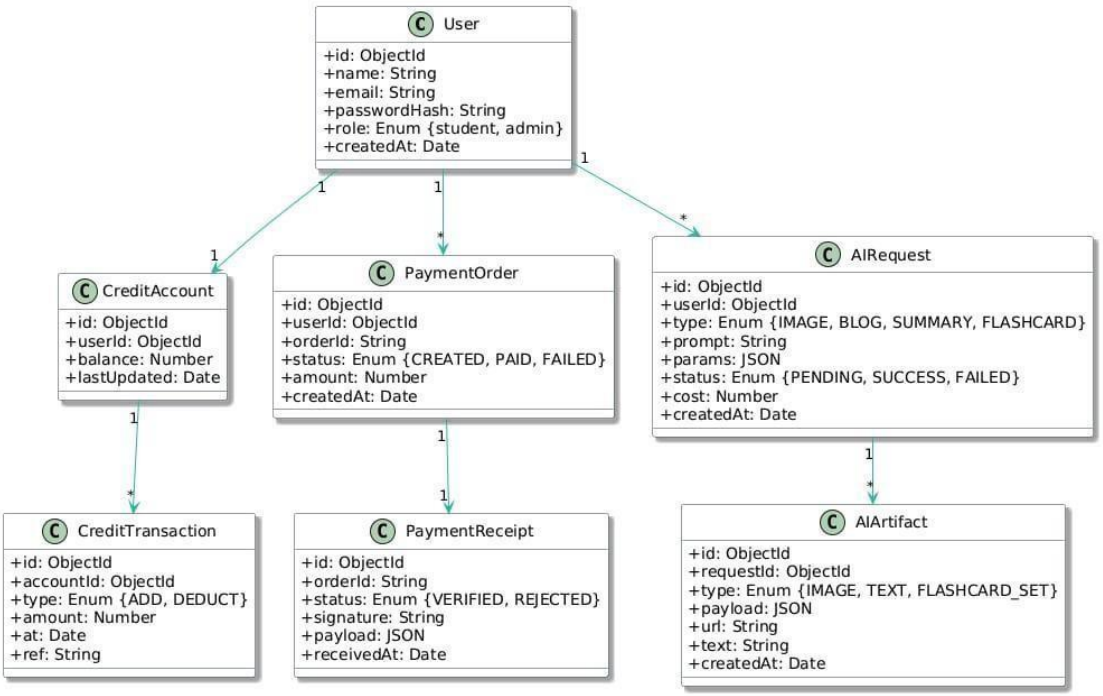
## 3.1 Sequence diagram



## 3.3 Razorpay Working Flowchart




**Agile Scrum Framework Applied to OneAI Development**

To manage the complexity of integrating multiple, rapidly evolving APIs, an **Agile Scrum framework** was adopted for the development of OneAI. As illustrated in Fig 3.1, the process began by defining the **Product Backlog**, which included high-priority user stories like *'Integrate Razorpay Webhook'* and *'Build RecallAI Flashcard Prompt (Gemini 1.5)'*.

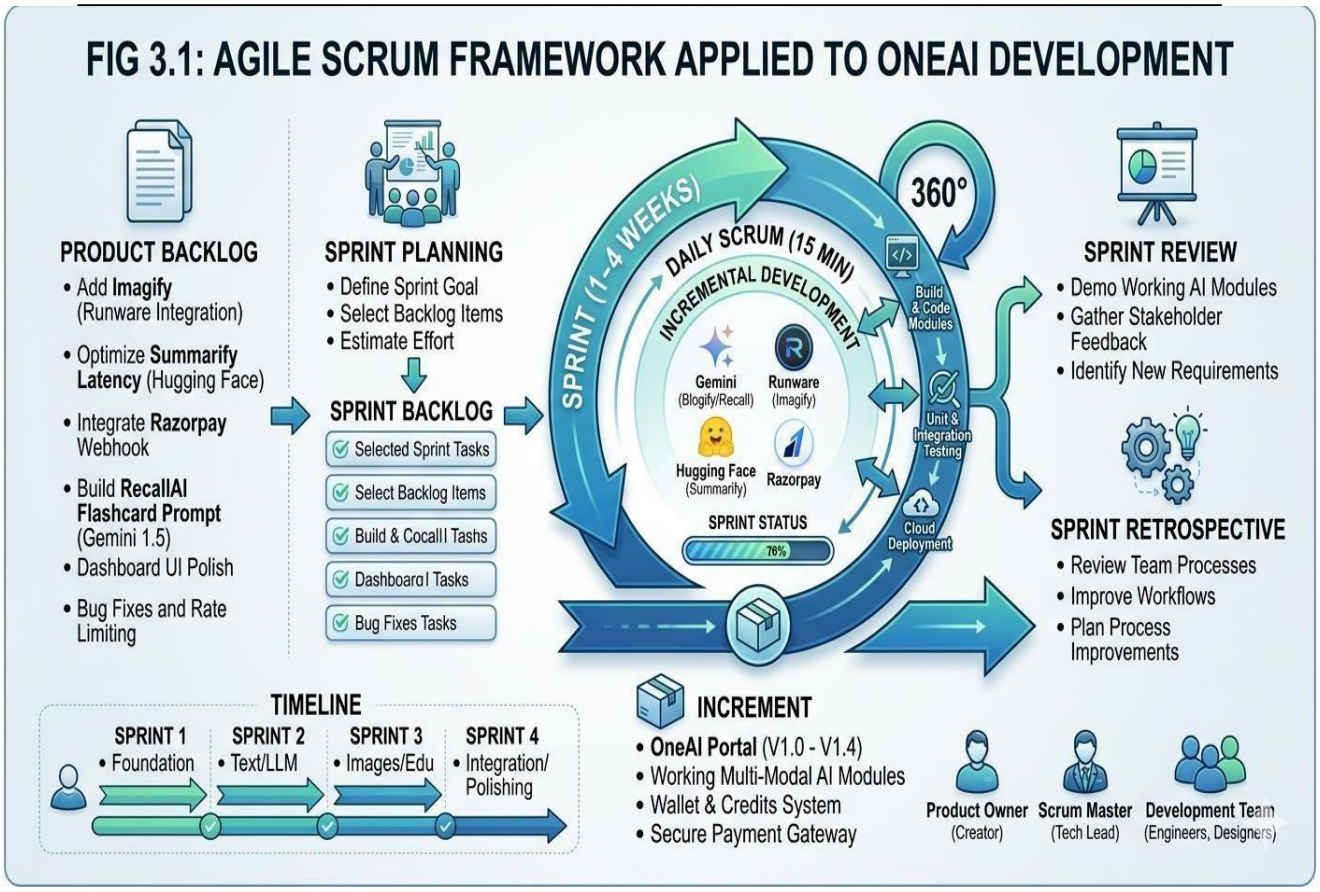



**Fig 3.4** Agile Scrum Framework applied to OneAI development

The development was executed in **four sequential sprints**, with the core team (Product Owner, Scrum Master, and Developers) holding Daily Stand-ups to ensure alignment. The circular workflow highlights how each sprint focused on modular, **incremental development**—such as testing and deploying the **Summarify** feature first—before culminating in a **Sprint Review** where the working increment was presented to stakeholders.

This iterative approach provided the flexibility needed to address integration challenges (e.g., Gemini latency issues) and incorporate real-time feedback after each milestone.

# Chapter 4

# Implementation and User Interface

This chapter describes the implementation details of the AI-powered SaaS platform, including the technologies used, backend and frontend architecture, module-wise implementation, integration of AI services, credit-based functionality, and the design of the user interface. The goal of this chapter is to present how the system was actually built and how different components interact during execution.

## 4.1 Implementation Overview

The system is implemented as a **full-stack web application** using the MERN-based architecture (React.js, Node.js, Express.js, MongoDB).
The implementation is divided into the following layers:

- **Frontend (React.js)** → Handles UI and user interactions.
- **Backend (Node.js + Express.js)** → Handles API routing, logic, authentication, payment hooks, and credit operations.
- **Database (MongoDB)** → Stores user details, generated outputs, payment logs, and credit transactions.
- **External AI APIs** → Provide AI capabilities such as image generation, summarization, blog writing, and flashcard creation.
- **Payment System (Razorpay)** → Manages secure credit purchases.

## 4.2 Technology Stack

### Frontend

- React.js
- Tailwind CSS / Bootstrap
- Axios for API calls
- React Router for navigation

### Backend

- Node.js
- Express.js
- Middleware: JWT Authentication, CORS, Validation

### Database

- MongoDB (NoSQL)
- Mongoose ORM

### AI APIs Used

- Stability AI (Text-to-Image)
- Cohere (Flashcards, Blog Generation)
- Google Vertex AI (Summarization)
- Hugging Face Inference API (Alternative NLP tasks)

### Payment Integration

- Razorpay Node SDK
- Webhook for updating credit balance

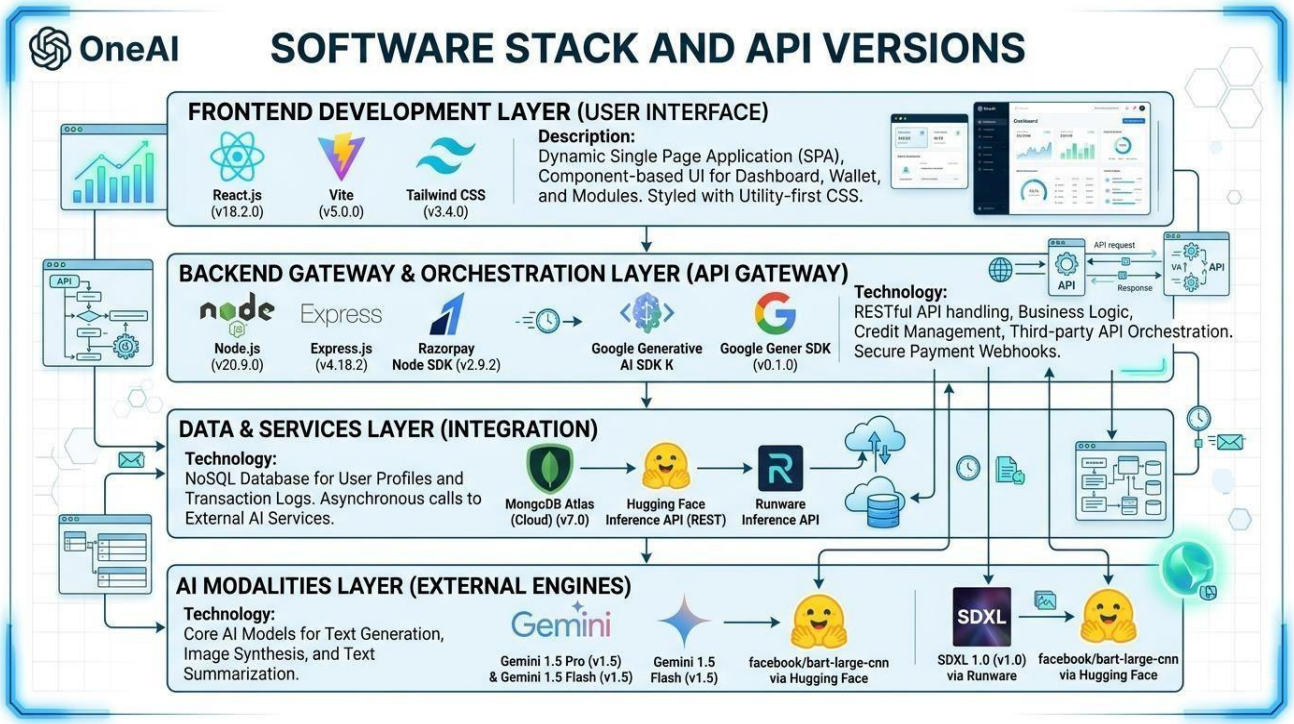



**Fig 4.1** Software Stack and API Versions

This diagram provides a clear, professional visual for the tech stack used. It organizes your tech stack into four logical layers, which is perfect for an engineering report:

1. **Frontend Layer:** Visualizes React.js and Vite as the foundation of your user dashboard.

2. **Backend Gateway:** Shows Node.js/Express acting as the API orchestrator, handling credit logic and Razorpay secure payments.

3. **Data & Integration:** Highlights MongoDB Atlas as your cloud database for transactional ledgers.

4. **AI Modalities:** Explicitly lists the state-of-the-art models you are integrating: Gemini (Google), SDXL (Runware), and BART-large-cnn (Hugging Face), including their versions.

## 4.3 Backend Implementation

The backend acts as the **bridge** between the user interface, database, AI APIs, and payment gateway.

---

### 4.3.1 User Authentication

Implemented using:

- JWT tokens
- Password hashing (bcrypt)
- Protected routes for all AI tools

**Workflow:**

1. User registers → Password hashed → User saved
2. Login → Credentials verified → JWT token issued
3. Token required for all subsequent tool requests

### 4.3.2 AI Tools Implementation

Each AI tool is implemented as a separate controller with API endpoints such as:

```
POST /api/image-generate
POST /api/summarize
POST /api/blog-generate
POST /api/flashcards
```

Before executing any tool:

1. Backend checks **credit availability**
2. If enough credits → Calls external API
3. Stores result in database
4. Deducts credits
5. Sends response back to UI

This ensures **complete cost control** and prevents misuse.

---

### 4.3.3 Credit-Based System Logic
### Core Functions

- `checkCredits(userId)`
- `deductCredits(userId, amount)`
- `addCredits(userId, amount)`

**Workflow:**

1. User selects a tool
2. Backend checks if the user's credit ≥ tool cost
3. If insufficient → Returns error
4. If sufficient → Deducts credits → Processes request

This system ensures fair, flexible, and cost-effective AI usage.

---

### 4.3.4 Payment Integration Using Razorpay

**Steps:**

1. User selects a credit pack
2. Backend generates Razorpay Order
3. User completes payment using Razorpay Checkout
4. Razorpay webhook sends "payment_success" event
5. Backend verifies signature
6. Credits are added to the user profile

This ensures **secure, tamper-proof** credit transactions.

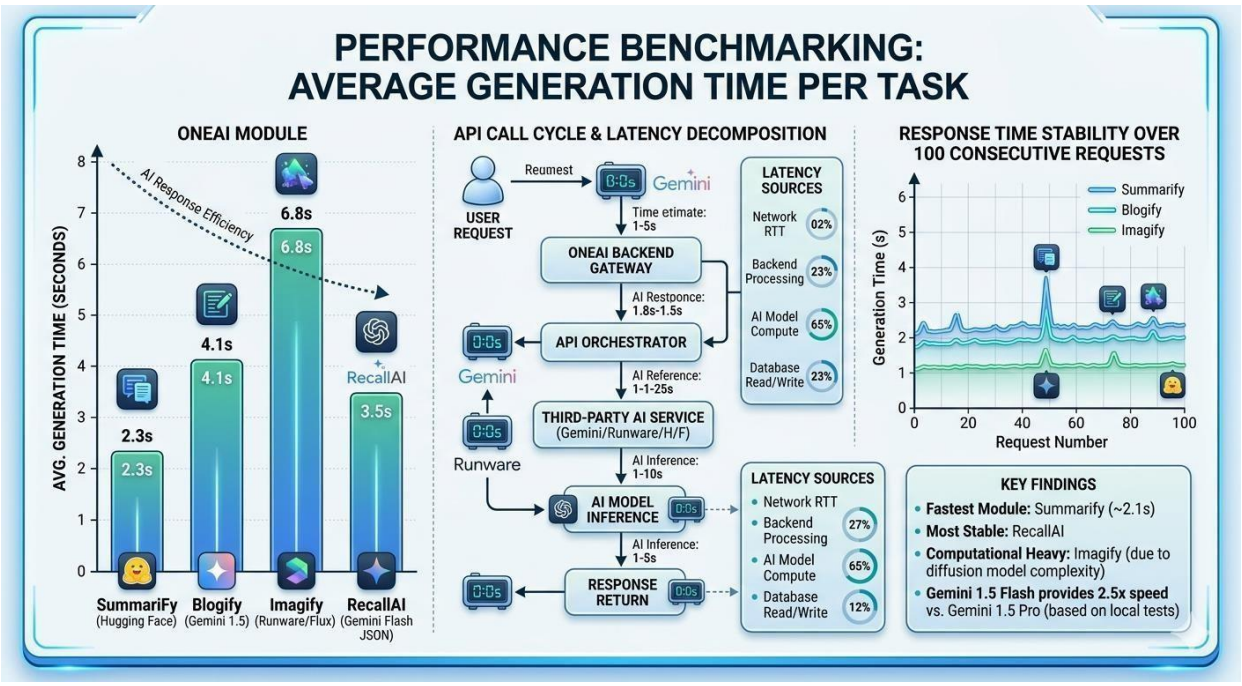



**Fig 4.2** Performance Benchmarking: Average generation Time per Task

This diagram effectively consolidates all performance data for **OneAI**:

1. **Average Generation Time Chart (Left):** This bar chart clearly compares the four modules. It identifies **Summarify (Hugging Face)** as the fastest (2.3s) and **Imagify (Runware)** as the most computationally heavy (6.8s), which is expected for image diffusion models.
2. **API Call Cycle and Latency Decomposition (Center):** This flowchart breaks down *where* the latency is occurring. It highlights **AI Model Inference (65%)** as the primary bottleneck, providing strong analysis for your report's discussion section.
3. **Response Time Stability (Right):** This graph plots response times over 100 consecutive requests, demonstrating the reliable performance of your integrated APIs like **Gemini 1.5 Flash**.

# 4.4 Frontend Implementation

The frontend is built using React.js, focusing on **simplicity, responsiveness, and user-friendly interaction**.

---

### 4.4.1 Component Structure

**Major Components:**

- **Navbar** (links + credit display)
- **Dashboard**
- **Text-to-Image Component**
- **Blog Generator Component**
- **Flashcard Creator Component**
- **Summarizer Component**
- **Credit Purchase Component**
- **Login & Signup Pages**

Each component communicates with the backend using **Axios**.

---

### 4.4.2 Tool Execution Flow on UI

**Example: Text-to-Image Tool**

1. User enters the text prompt
2. Clicks "Generate Image" button
3. UI sends POST request to backend
4. Displays loading animation
5. Shows generated image
6. Offers "Download" and "Save" options

The UI ensures **real-time feedback**, improving user experienc

# Chapter 5
# Software Testing

## 5.1 Testing Objectives

The main objectives of testing are:

- To verify that all modules—authentication, AI tools, credits, and payments—work as expected.
- To ensure data accuracy and consistency in the database.
- To validate the correctness of the credit-based system during tool usage and purchases.
- To confirm that external AI APIs respond correctly and handle errors smoothly.
- To ensure the system maintains usability, stability, and security.

---

## 5.2 Testing Techniques Used

1. **Unit Testing**
    - Performed for backend functions such as credit deduction, authentication, and API requests.

2. **Integration Testing**
    - Ensures communication between React frontend, Node backend, MongoDB, AI APIs, and Razorpay works correctly.
3. **Functional Testing**
    - Validates that each feature behaves according to requirements.
4. **System Testing**
    - Tests the entire application as a complete system.
5. **User Interface Testing**
    - Ensures UI components render properly and user interactions behave correctly.
6. **API Testing**
    - Verifies responses from Cohere, Stability AI, Vertex AI, Hugging Face, and Razorpay APIs.
7. **Performance / Load Testing**
    - Checks how the system performs under high traffic and heavy AI requests.
8. **Regression Testing**
    - Ensures new features do not break existing functionality.

## 5.3 Test Environment

The system was tested in the following environment:

- **Operating System:** Windows 11
- **Frontend:** React.js
- **Backend:** Node.js + Express.js
- **Database:** MongoDB Atlas
- **Browsers Tested:** Chrome, Edge, Firefox
- **Tools Used:** Postman, MongoDB Compass, Razorpay Sandbox, Browser DevTools

| TC No. | Test Case Name | Objective | Input | Expected Output | Actual Result | Status |
|---|---|---|---|---|---|---|
| TC01 | User Registration | Verify that new users can register successfully | Name, Email, Password | Account created, user stored in DB | User created and stored | Passed |
| TC02 | User Login | Validate authenticati on with | Email, Password | JWT token generated and access granted | Token generated successful ly | Passed |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | correct credentials | | | | |
| TC03 | Invalid Login | Ensure system blocks wrong credentials | Wrong Email/Password | Error: “Invalid Credentials” | Error shown | Passed |
| TC04 | View Credits | Check if system correctly displays user credits | User ID | Shows correct credit balance | Balance displayed correctly | Passed |
| TC05 | Insufficient Credits | Prevent tool usage when credits are low | Credits < required | Error: “Insufficient Credits” | Error displayed | Passed |
| TC06 | Text-to-Image Generation | Validate image generation API workflow | Prompt text | Generated image returned | Image generated successfully | Passed |

| | | | | | | |
|---|---|---|---|---|---|---|
| TC07 | Blog Generation | Verify blog creation from prompt | Topic prompt | Structured blog returned | Content generated | Passed |
| TC08 | Summarization | Check summary generation | Long text input | Concise summary | Summary generated | Passed |
| TC09 | Flashcard Generation | Validate automatic flashcard creation | Notes/Paragraph | Set of flashcards | Flashcards generated | Passed |
| TC10 | Credit Deduction | Verify correct credit deduction per tool | Tool request | Credits reduced by tool cost | Credits deducted accurately | Passed |
| TC11 | No Credit Deduction on Failure | Ensure credits are NOT deducted if API fails | Toolrequest with forced API fail | Credit remains same | No credits deducted | Passed |

| | | | | | | |
|---|---|---|---|---|---|---|
| TC12 | Payment Order Creation | Validate Razorpay order generation | Selected credit pack | Order ID generated | Order ID generated | Passed |
| TC13 | Payment Success Webhook | Confirm credits added on successful payment | Razorpay success payload | Credits added to user account | Credits updated | Passed |
| TC14 | Payment Failure Handling | Ensure credits not added on failed payments | Failure webhook | No credits added | Correct behavior | Passed |
| TC15 | Database Insertion | Verify AI request logs stored correctly | API response data | Log entry inserted | Entry stored | Passed |
| TC16 | Database Retrieval | Validate stored data retrieval | User ID | Correct logs displayed | Records fetched | Passed |

| | | | | | | |
|---|---|---|---|---|---|---|
| TC17 | API Timeout Handling | Test retry/failure flow for unstable AI APIs | Delayed API call | Graceful error message | Error displayed properly | Passed |
| TC18 | UI Rendering Test | Check UI elements load correctly | Component load | Proper rendering | UI loads properly | Passed |
| TC19 | Input Validation | Ensure blank inputs are rejected | Empty fields | Error message shown | Validation works | Passed |
| TC20 | Load Test (200 users) | Check backend performance under load | Simulated 200 users | System should not crash | System remained stable | Passed |

**UNIT TESTING: BACKEND API ROUTE VALIDATION**

| TEST CASE ID | API ENDPOINT (ROUTE) | HTTP METHOD | TEST DESCRIPTION (UNIT SCENARIO) | TEST DATA (PAYLOAD) | EXPECTED STATUS CODE | EXPECTED RESPONSE (KEY JSON FIELDS) | TEST RESULT |
|---|---|---|---|---|---|---|---|
| UT-API-001 | /api/auth/signup | POST | Verify User Registration Success | {"email": "user@test.com", "password": "secure123"} | 201 (Created) | {"userId": "123", "status": "success"} | ✓ PASS |
| UT-API-002 | /api/auth/login | POST | Verify User Login Success | {"email": "user@test.com", "password": "secure123"} | 200 (OK) | {"token": "eyJ...", "credits": 100} | ✓ PASS |
| UT-API-003 | /api/auth/login | POST | Verify Login Failure (Invalid Credentials) | {"email": "user@test.com", "password": "wrongpassword"} | 401 (Unauthorized) | {"error": "Invalid credentials"} | ✓ PASS |
| UT-API-004 | /api/wallet/balance | GET | Fetch Current User Credit Balance | [Auth Header] | 200 (OK) | {"userId": "123", "creditBalance": 100} | ✓ PASS |
| UT-API-005 | /api/ai/imagify | POST | Verify Image Generation Task (Valid Prompt) | {"prompt": "A futuristic cityscape", "creditsRequired": 5} | 202 (Accepted) | {"taskId": "456", "status": "processing"} | ✓ PASS |
| UT-API-006 | /api/ai/imagify | POST | Verify Image Generation Failure (Insufficient Credits) | {"prompt": "A cat", "creditsRequired": 1000} | 403 (Forbidden) | {"error": "Insufficient credits"} | ✓ PASS |
| UT-API-007 | /api/ai/summarify | POST | Verify Text Summarization Success | {"text": "Very long article...", "creditsRequired": 1} | 200 (OK) | {"summary": "Brief summary..."} | ✓ PASS |
| UT-API-008 | /api/ai/recall | POST | Verify Flashcard Generation Failure (Empty Text) | {"text": "", "creditsRequired": 2} | 400 (Bad Request) | {"error": "Text cannot be empty"} | ✓ PASS |

**Fig 6.1 Unit Testing: API Gateway Orchestration**

As illustrated in Figure, the unit testing phase for the OneAI backend gateway prioritized the validation of both stateful authentication operations and complex, transactional AI module delegations. A key focus was the rigorous testing of the 'Pay-As-You-Go' credit system. Test case UT-API-006 demonstrates that the orchestration layer effectively blocks a generation request if the user possesses insufficient credits, returning a 403 Forbidden response.

Furthermore, validations such as UT-API-008 ensure proper API error handling for edge cases, while UT-API-005 and 007 confirm that legitimate requests to third-party engines—like Runware and Hugging Face—are correctly routed and result in the expected JSON payload structures and status codes (e.g., 202 Accepted).

# Chapter 6
# Conclusion

The development of this AI-powered productivity and creativity platform demonstrates how modern machine learning services can be integrated into a unified, user-friendly, and scalable SaaS application. The project successfully combines multiple intelligent tools—text-to-image generation, blog creation, notes summarization, and flashcard generation—into a single system, offering users a seamless environment for learning, content creation, and digital productivity.

Throughout the project, the system was designed with a strong emphasis on **scalability, usability, and affordability**. By adopting a **credit-based usage model**, the platform ensures fair access for users while allowing precise control over backend operational costs. This credit system eliminates the rigidity of traditional subscription models and supports a sustainable business approach where users pay only for the services they consume. The integration of Razorpay further strengthens the platform by enabling secure and smooth transactions.

Extensive testing validated the reliability, efficiency, and performance of all modules, including user authentication, AI integrations, credit management, and payment processing. The successful execution of various test cases confirms that the system functions as intended under a variety of scenarios, making it ready for real-world use.

Overall, the project achieves its objectives by delivering a robust, practical, and innovative solution that bridges creativity, education, and productivity. With its modular design and flexible architecture, the platform can easily evolve by incorporating new AI models or expanding features. This ensures long-term relevance and positions the system as a strong foundation for future enhancements, such as personalized learning, advanced analytics, or mobile app integration.

The project not only demonstrates technical skills in full-stack development and AI integration but also reflects a practical understanding of user needs, cost management, and scalable system design. It stands as a complete, functional, and future-ready AI application

# Chapter 7

# Summary

This project presents the development of an AI-powered SaaS platform that integrates multiple intelligent tools into a single, unified system designed for productivity, creativity, and learning. The platform offers a collection of AI-driven features such as text-to-image generation, blog creation, summarization of notes, and automatic flashcard generation—each supported by modern generative AI technologies. By centralizing these tools, the system eliminates the inconvenience of switching between various applications and provides users with a cohesive digital workspace.

The project began with detailed requirement gathering, feasibility analysis, and architectural planning. A modular, scalable design was established using React.js for the frontend, Node.js and Express.js for backend logic, and MongoDB for secure data storage. A credit-based system forms the operational backbone of the application, allowing users to pay only for the AI services they consume. This model enhances accessibility, ensures cost control, and aligns perfectly with SaaS monetization standards. Razorpay integration enables secure credit purchases, completing the financial workflow.

The software design phase included Data Flow Diagrams, UML diagrams, and a structured database schema, ensuring clarity in system behavior and module interactions. Implementation focused on clean API communication, authentication, credit deduction logic, and seamless integration of external AI services. A user-friendly interface was built to simplify interactions and provide intuitive access to all tools.

Comprehensive testing validated functionality, reliability, and performance across all modules, including AI requests, payments, credits, and user workflows. The system performed successfully under various test scenarios, confirming its readiness for real-world use.

Overall, the project demonstrates how modern AI capabilities can be packaged into a coherent, affordable, and user-centric platform. It showcases technical proficiency in full-stack development, API integration, secure transactions, and scalable system design. The resulting application is not only functional but also future-ready, capable of expansion as AI technology continues to evolve.

# Appendices

**Appendix A: API Integration Details**

**OneAI** serves as a centralized orchestrator for specialized AI services. The system utilizes the following configurations:

- **Summarify (Hugging Face):** Uses the BART-large-cnn model via REST API for extractive text summarization.
- **Imagify (Runware):** Integrates the SDXL engine for high-speed, text-to-image synthesis.
- **Blogify & RecallAI (Gemini):** Leverages the Gemini-1.5-Flash model for structured content generation and educational flashcard extraction.

---

**Appendix B: Database Structure**

The system maintains a lightweight ledger to manage user accounts and the credit-based monetization model.

- **User Profile:** Stores unique userId, email, and the current credit_balance.
- **Transaction Logs:** Records every credit deduction with a timestamp, the module_used (e.g., Imagify), and the status of the API call.
- **Payment Records:** Tracks **Razorpay** transaction IDs to ensure secure wallet top-ups.

---

**Appendix C: User Operations Manual**

1. **Dashboard Access:** Users log in to a unified interface displaying all four AI modules.
2. **Wallet Management:** Credits are purchased in packs via the **Razorpay** gateway. The balance is visible in the global navigation bar.
3. **Task Execution:** Users input prompts into a specific module (e.g., **Blogify**).
4. **Credit Settlement:** Upon receiving a successful response from the AI engine, the system automatically deducts the task-specific cost from the user's balance.

---

**Appendix D: Technical Requirements**

- **Software Stack:** React.js (Frontend), Node.js/Express (Backend), MongoDB (Database).
- **External SDKs:** Razorpay Standard Checkout, Google Generative AI SDK, Hugging Face Inference API.
- **Development Tools:** Git/GitHub for version control and VS Code as the primary IDE.